

Genus 2 Curve Arithmetic System

Technical Overview

Abstract

Rational point search · Jacobian rank computation · Finite-field DLP.

1 Executive Summary

This system is a research-grade SageMath/Python toolkit for studying the arithmetic of genus 2 (and higher-genus) hyperelliptic curves over the rationals. Its central innovation is an iterated fibration tower construction that reduces difficult questions about genus ≥ 2 curves to tractable problems on families of elliptic curves. Building on this foundation, the system provides three tightly integrated capabilities:

- Rational point enumeration — finding rational points on curves of arbitrary genus (genus 2 fully supported; genus 3 tested; higher genera possible) starting from a single seed point, using CRT, LLL lattice reduction, and Hensel lifting.
- Jacobian rank determination (`MUMFORD_SEARCH` mode) — computing a provably independent basis for $J(\mathbb{Q})$, the Mordell–Weil group of the Jacobian, typically in well under two minutes regardless of conductor or rank.
- Finite-field DLP / index calculus — mounting a genus 2 hyperelliptic-curve discrete-logarithm attack via Mumford divisor smoothness relations and sparse linear algebra.

The system has been validated against the LMFDB on dozens of genus 2 curves spanning a wide range of conductors (up to $\sim 10^7$) and Mordell–Weil ranks (0 through $4+$).

2 Mathematical Background

2.1 Genus 2 Curves and Their Jacobians

A genus 2 curve C over $\mathbb{Q}$ is a smooth projective curve of the form $y^2 = f(x)$ where f is a squarefree polynomial of degree 5 or 6. Unlike elliptic curves, $C(\mathbb{Q})$ need not itself form a group; instead, its degree-0 Picard group — the Jacobian $J(C)$ — is a 2-dimensional abelian variety that generalizes the group law. By the Mordell–Weil theorem, $J(\mathbb{Q})$ is a finitely generated abelian group. Its free rank $r = \text{rank } J(\mathbb{Q})$ (often just called the rank of C) is a central arithmetic invariant. Computing r rigorously is much harder for genus 2 than for elliptic curves, where standard descent methods apply directly.

Elements of $J(\mathbb{Q})$ are represented as Mumford divisors: pairs $(u(x), v(x))$ where u is monic of degree 2, v has degree < 2 , and $u \mid v^2 - f(x)$ in $\mathbb{Q}[x]$. The Cantor algorithm handles group-law computations in this representation.

2.2 Elliptic Fibrations

An elliptic fibration is a map $\pi : S \rightarrow \mathbb{P}^1$ from a surface S to the projective line whose generic fiber is a smooth elliptic curve $E/\mathbb{Q}(m)$. The system constructs such a fibration starting from the genus 2 curve C and a rational point on C . The parameter m lives in the intersection locus: it records where the tangent (or secant) line through a known rational point meets the shifted curve. A rational m -value corresponds to a rational point on C , which is precisely the information the search harvests via CRT.

2.3 Shioda–Tate and Picard Numbers

For an elliptic surface $S/\mathbb{Q}$ with generic fiber $E/\mathbb{Q}(m)$, the Shioda–Tate formula relates the Picard number $\rho(S)$, the rank of $E(\mathbb{Q}(m))$, and the fiber contribution:

$$\rho(S) = 2 + \text{rank } E(\mathbb{Q}(m)) + \sum_v (m_v - 1)$$

where the sum is over singular fibers of Kodaira type, and m_v is the number of irreducible components. The system computes both a lower bound on ρ (from characteristic-0 geometry) and an upper bound (from reduction modulo good primes via the Lefschetz trace formula), often pinning ρ exactly and thereby determining the Mordell–Weil rank of the fibration.

3 Core Algorithm: The Fibration Tower

3.1 Tower Construction

Given a genus 2 curve $C : y^2 = f(x)$ and a seed rational point $P = (x_0, y_0) \in C(\mathbb{Q})$, the system constructs an iterated fibration tower:

- Degree 5 polynomials (genus 2, one infinite place): one tower step reduces to a degree-4 (genus 1) quartic fibration.
- Degree 6 polynomials (genus 2, two infinite places): two tower steps are required, first reducing to degree 5, then to degree 4.

At each step the algorithm:

1. Selects the best-scoring auxiliary polynomial $Q(x)$ of appropriate degree using a complexity measure (height, degree, collision resistance of the discriminant locus).
2. Computes the intersection locus as a rational function $r(m)$ in the fibration parameter m .
3. Passes to the next step using the residual degree-4 quartic as the new fiber.

Scores are computed across 10 candidate geometries and the lowest-complexity choice is selected, trading off coefficient height, degree, and bad-prime content. Non-minimal models are sometimes preferred despite seemingly higher complexity, because they are less sensitive to height bounds.

3.2 Weierstrass Model and Fiber Analysis

Once the quartic fiber is established, the system computes the Weierstrass model $E : y^2 = x^3 + a_4(m)x + a_6(m)$ over the function field $\mathbb{Q}(m)$. This involves:

- Minimal Weierstrass reduction: Tate’s algorithm applied fiber by fiber over algebraic places of m .

- Kodaira fiber classification: each irreducible factor of the discriminant $\Delta(m)$ is classified ($I_n, II, III, IV, I_n^*, \dots$) by examining the valuations of c_4, c_6, Δ .
- Euler characteristic validation: the sum of fiber contributions must equal 12 for a standard rational elliptic surface.
- Discriminant degree check: degree 12 in m indicates a fully minimal model; deviations trigger diagnostics.

3.3 Mordell–Weil Sections

From the known rational point P , the system generates sections — rational functions $S_i(m) = (x_i(m), y_i(m))$ satisfying the Weierstrass equation for all m . Sections are the Mordell–Weil group elements of the generic fiber. Multiple independent sections arise from multiples $[n]P$ of the base section and from additional rational points found during the search. Independence is verified via the Néron–Tate height pairing matrix, which must be positive definite for a true basis.

4 Rational Point Search over $\mathbb{Q}$

4.1 The Search Problem

The key observation is that a rational point $(x_0, y_0) \in C(\mathbb{Q})$ corresponds to a rational value of m satisfying $x_0 = r(m)$ where $r(m)$ is the intersection-locus rational function. The search therefore reduces to finding rational roots of the numerator of $S_i(m) - r(m)$ for sections S_i .

4.2 Modular Arithmetic Pipeline

The system searches for rational m by collecting residues modulo a pool of primes and applying CRT reconstruction:

1. For each prime p in the pool, solve $S_i(m) \equiv r(m) \pmod{p}$ to find candidate residues $m \equiv c \pmod{p}$.
2. Apply Hensel’s lemma to lift simple roots to higher precision.
3. Filter by torsion order (small-order residues likely correspond to torsion sections, not rational points).
4. Combine residues across multiple prime subsets via CRT, forming candidate rational numbers by rational reconstruction.
5. Verify each candidate by substituting back into the curve equation.

4.3 LLL Lattice Reduction

Raw search vectors (integer combinations of sections) are preprocessed by LLL and BKZ lattice reduction to find short vectors. Short vectors correspond to low-multiplier combinations $[n]P$ that are most likely to produce small-height rational m values, dramatically reducing the search space. Configurable parameters include the height bound, prime pool size, LLL delta, BKZ block size, and extra-prime filtering density. The system auto-tunes several of these from the Galois group of the discriminant and the canonical height of the seed point.

4.4 Multi-Fibration Strategy

Different fibrations — built from different auxiliary polynomials $Q(x)$ — expose different rational points because each section sees the curve’s geometry from a different angle. Not every fibration can detect every rational point. The system therefore runs multiple fibrations sequentially, adding newly discovered points as seeds for subsequent fibration towers. A consensus filter can optionally require agreement across several fibrations before accepting a candidate.

5 Jacobian Rank Finding: MUMFORD_SEARCH Mode

This mode directly computes the Mordell–Weil rank of $J(C)$ by constructing an explicit independent basis for $J(\mathbb{Q})$. It runs in well under 2 minutes for genus 2 curves of arbitrary conductor and rank.

5.1 What It Computes

Whereas the rational point search looks for rational m -values (i.e., rational points on C), MUMFORD_SEARCH mode looks for rational Mumford divisors: elements $D \in J(\mathbb{Q})$ of the form $[u(x), v(x)]$ with $u, v \in \mathbb{Q}[x]$, $\deg u = 2$, $\deg v < 2$. The size of the largest independent such set is the Mordell–Weil rank.

5.2 The Mumford Residue System

Each Mumford divisor is specified by four rational numbers (s, p, v_1, v_0) , where $u(x) = x^2 - sx + p$ and $v(x) = v_1x + v_0$. The Mumford conditions give a system of polynomial constraints. The fibration provides a modular parameterization: for each prime ℓ in the pool, the system solves the 5-equation Mumford system over $\mathbb{F}_\ell$ to harvest candidate residues of (s, p, v_1, v_0) .

5.3 CRT Reconstruction of Divisors

Residues of the four Mumford coordinates are collected across the prime pool and combined via CRT. The reconstruction phase:

- Groups vectors by their LLL-reduced section multiple, focusing reconstruction on vectors most likely to lift to rational divisors.
- Applies algebraic pre-filtering using the consistency equations to prune incompatible residue combinations before CRT.
- Performs height-filtered rational reconstruction, accepting only candidates whose Weil height is within the configured bound.
- Verifies each reconstructed (s, p, v_1, v_0) against the Mumford conditions over $\mathbb{Q}$.

5.4 Independent Basis Construction

After reconstruction, the system selects an independent basis from the candidate divisors using a multi-prime certification protocol:

- Torsion filter: divisors with finite order in $J(\mathbb{Q})$ are identified by checking $[N]D = 0$ for small N using the group law, and discarded.
- Mod- p scoring: each divisor is evaluated modulo several good primes; those with consistently high “rank score” across primes are prioritized.

- Incremental certification: divisors are added to the basis one at a time. A new divisor is accepted only if it is independent of the current basis modulo at least 3 distinct primes. Independence is checked via Smith normal form on the relation matrix.
- Height pairing confirmation: once the basis is certified by mod- p methods, the Néron–Tate Gram matrix $H = (\langle D_i, D_j \rangle)$ is computed using Arakelov height pairings and confirmed to be positive definite.

The rank is simply the final basis size. Because the certification uses provably correct linear algebra over $\mathbb{Z}/n\mathbb{Z}$ (Smith normal form) at multiple primes, the result is certified rather than heuristic.

5.5 Typical Performance

For genus 2 curves from the LMFDB with conductors up to $\sim 10^7$ and Mordell–Weil ranks 0 to 4, the complete rank-finding computation — fibration construction, residue harvesting, CRT reconstruction, torsion filtering, and basis certification — typically completes in under 2 minutes on a modern multicore workstation. The bottleneck is the CRT reconstruction phase, which is parallelized across up to 20 workers.

As an example, a rank-4 curve ($y^2 = x^6 + 8x^5 + 10x^4 - 10x^3 - 11x^2 + 2x + 1$) with 11 known rational points: the system found a certified basis of 4 independent Mumford divisors and all 7 non-trivial rational x -coordinates in 33.5 seconds total.

6 Finite-Field Index Calculus (DLP Mode)

6.1 Problem Setup

When `FINITE_FIELD = p` is set, the system attacks the genus 2 Hyperelliptic Curve Cryptosystem (HECC) discrete logarithm problem: given $G, Q \in J(\mathbb{F}_p)$ with $Q = [d]G$, find d . The system automatically generates a cryptosystem instance: it constructs a prime-order subgroup of $J(\mathbb{F}_p)$, picks a random base divisor G , computes $Q = [d]G$ for a chosen secret key d , and records four preferred x -coordinates (the supports of G and Q) to aid smoothness detection.

6.2 Index Calculus Pipeline

The attack proceeds in three stages:

1. Factor base construction: smooth Mumford divisors are collected using the fibration-parameterized Mumford residue system over $\mathbb{F}_p$. “Smooth” means $u(x)$ splits completely over $\mathbb{F}_p$ into linear factors, giving degree-1 components that form the factor base.
2. Relation generation: the random walk harvests divisors expressible as sums of factor-base elements. A structured walk (seeded by x -residues from the fibration) and a collision walk (Baby-step Giant-step) cooperate to reach the target relation count.
3. Linear algebra: the relation matrix (sparse, over $\mathbb{Z}/\ell\mathbb{Z}$ where ℓ is the group order) is solved using Block Wiedemann’s algorithm. The solution gives d directly.

6.3 Matrix Rank Diagnostics

Before committing to linear algebra, the system verifies that the relation matrix is full rank. A full-rank matrix guarantees a unique solution; rank deficiency indicates insufficient or linearly dependent relations and triggers further relation collection.

7 Supporting Analyses

7.1 Picard Number (ρ) via Shioda–Tate

Using the Shioda–Tate formula, the system computes both a characteristic-0 lower bound on $\rho(S)$ (from the known sections and fiber types) and an upper bound from the rank of the Néron–Severi group modulo a good prime ℓ (via the Lefschetz trace on the ℓ -adic cohomology). When both bounds agree, ρ is determined exactly, which in turn pins the Mordell–Weil rank of the generic fiber.

7.2 2-Selmer Rank Bounds

A 2-descent computes an upper bound on $\text{rank } J(\mathbb{Q})$ via the Selmer group. Local conditions are evaluated at each bad prime (including ∞) and the global Selmer rank is bounded by the sum of local dimensions minus the reciprocity correction. This provides an upper bound complementing the lower bound from the Mumford basis.

7.3 Torsion Analysis

The system identifies the torsion subgroup of $J(\mathbb{Q})$ by computing the GCD of specializations of the group order at good primes and by checking small multiples $[N]D = 0$. Torsion elements are filtered out before basis construction to ensure the Mumford basis is free.

7.4 Automorphism Analysis

The Néron–Severi lattice automorphisms (scaling and translation symmetries of the fibration) are computed and reported. Non-trivial automorphisms can be exploited to reduce the search space or to relate different fibrations.

7.5 CM Fiber Detection

Special fibers with complex multiplication (j -invariant $0, 1728, -19728, -88736, \dots$) are detected by solving $a_4(m) = 0$ or specialized j -invariant equations. CM fibers often yield high-rank specializations and are tried as candidate rational points.

7.6 Archimedean Height Pairings

The Néron–Tate height pairing $\langle D_i, D_j \rangle$ between Mumford divisors is computed using a combination of Arakelov intersection theory (for the finite places) and numerical integration over the Jacobian’s period lattice (for the archimedean place). These pairings form the Gram matrix H , whose determinant and positive-definiteness certify independence of the basis.

8 Feature Summary

9 Usage and Configuration

9.1 Primary Entry Point

The system is invoked via the main entry point. All configuration lives in the configuration file.

Feature	Description
Fibration Tower	Iterated 1–2 step tower reducing genus 2 curve to elliptic fibration over $\mathbb{Q}(m)$; best-of-10 geometry selection.
Weierstrass & Kodaira	Full Weierstrass minimization, Tate algorithm, Kodaira fiber classification, Euler characteristic.
Rational Point Search	LLL + CRT + Hensel + rational reconstruction pipeline; auto-configured height bounds and prime pool.
Rank Finding (MUMFORD)	Provably independent Mumford basis for $J(\mathbb{Q})$; mod- p certification + Smith normal form; under 2 min typical.
Torsion Filtering	Identifies and removes torsion Mumford divisors before basis construction.
Picard / Shioda–Tate	Exact $\rho(S)$ via char-0 lower bound + reduction upper bound; determines MW rank of generic fiber.
2-Selmer Bounds	Local 2-descent at all bad primes; upper bound on rank $J(\mathbb{Q})$.
CM Fiber Detection	Special j -invariant fiber search for high-rank specializations.
Automorphisms	NS lattice translation and scaling automorphisms; symmetry exploitation.
Finite-Field DLP	HECC index calculus: smooth divisor factory, structured + collision walk, Block Wiedemann solve.
Height Pairings	Arakelov + archimedean Néron–Tate pairings; positive-definite Gram matrix enforced.
Multi-Fibration	Sequential fibration construction from growing point list; consensus filter across fibrations.
Diagnostics	Completeness posterior, prime MI analysis, saturation ratio, sufficiency proofs.
Parallel Execution	Residue computation and CRT reconstruction parallelized across up to 20 workers.

Table 1: System Capabilities and Features

9.2 Key Configuration Flags

10 Representative Example Outputs

10.1 Rank-4 Curve — Full Point Set

- Curve: $y^2 = x^6 + 8x^5 + 10x^4 - 10x^3 - 11x^2 + 2x + 1$ (LMFDB “prestige curve”)
- Seed point: $(-1, 1)$
- MUMFORD_SEARCH mode: certified rank-4 Mumford basis in 33.5 seconds.
- 7 distinct rational x -coordinates recovered: $\{0, 1, -3/2, 2, 1/3, -2, -1\}$.
- 4 independent Mumford divisors certified by 4 primes, mod- p rank = 4.

10.2 Rank-1 Curve — Picard Number Determination

- Picard number $\rho = 3$ determined exactly (lower = upper = 3 at $\ell = 10009$).

Flag	Description
COEFFS_GENUS2	Coefficient list $[a_n, \dots, a_0]$ defining $y^2 = f(x)$. Degree 5 or 6 for genus 2; degree 7–8 for genus 3; higher degrees accepted (experimental).
DATA_PTS_GENUS2	List of seed x -coordinates (rational) on the curve.
TERMINATE_WHEN_6	Stop searching after finding this many distinct rational x -coordinates.
MUMFORD_SEARCH	True : find Mumford basis for $J(\mathbb{Q})$ (rank). False : find rational points on C . Default: False .
FINITE_FIELD	None for over- $\mathbb{Q}$ search; set to prime p for finite-field DLP mode. Default: None .
SECRET_KEY	DLP mode: secret exponent d (for benchmarking).
BLOCK_WIEDEMANN	True : always use Block Wiedemann for the final DLP solve. Default: False .
PRIME_POOL	Pool of primes for modular residue computation; defaults to <code>primes(590)</code> , auto-filtered per fibration.
NUM_PRIME_SUBSETS	Number of independent prime subsets; ≥ 250 for stability. Default: 500.
USE_MINIMAL_MODEL	True : Tate-minimized Weierstrass model (slower, more correct). False : generic fiber model. Default: True .
SYMBOLIC_SEARCH	True : solve over $\mathbb{Q}(m)$ (usually slower; rarely finds anything extra). Default: False .
HEIGHT_BOUND	Canonical height cutoff for rational reconstruction candidates. Auto-tuned by default.
DEBUG	Enable verbose diagnostic output. Default: False .

- Shioda–Tate: $\text{rank } E(\mathbb{Q}(m)) = 1$, $\sum (m_v - 1) = 0$.
- 2-Selmer bound: $1 \leq \text{rank} \leq 3$.
- Total run time: 125 seconds.

10.3 Finite-Field DLP

- Curve: $y^2 = x^5 + 3x^3 + 2x^2 + 5x + 4$ over $\mathbb{F}_{33554467}$
- Secret key $d = 800$, Jacobian order $\approx 10^{12}$, largest prime $\ell \approx 9.7 \times 10^8$.
- 18,017 smooth Mumford divisors collected (factor base of 4,504 unique x -coordinates).
- Relation matrix: 18017×4504 , full rank confirmed. Ready for Block Wiedemann.
- Total time: 112 seconds.

10.4 Large-Conductor Genus 2 Curves

The system has been tested on curves parsed directly from LMFDB database entries (discriminant up to $\sim 10^7$, conductors from hundreds to millions), with `TERMINATE_WHEN_6` targets of 1 to 11. Conductors up to $\sim 10^7$ are handled without special tuning.

11 Code Architecture

12 Known Limitations and Open Questions

- The 2-Selmer upper bound can greatly exceed the true rank when $\text{Sha}(J)[2]$ is large, giving a wide gap.
- Not every rational point is visible from a given fibration: some points may only appear via secondary fibrations or at very high section multiples.
- The Mumford basis gives a lower bound on rank, but proving it is the full rank requires the Selmer upper bound to match (or an unconditional height-bound argument).
- Local Mumford solutions in characteristic p do not always lift globally; the CRT machinery harvests only those that happen to glue, which remains a theoretical mystery.
- Higher genus support: genus 3 rational point search has been tested successfully via the tower construction, but the `MUMFORD_SEARCH` and DLP modes currently only support genus 2. Higher genera (≥ 4) are largely untested due to difficulty sourcing suitable test curves.
- Multi-point fibrations (2+ seed points) are not fully implemented and may hang.
- Very high-degree fibrations ($\text{deg} > 15$) may require increased `MAX_MODULUS`.
- The posterior completeness estimator is acknowledged to be unreliable and should not be used to conclude that the search has terminated.

13 Dependencies

- SageMath $\geq 10.x$ — core algebraic geometry, Jacobian arithmetic, LLL, CRT.
- Python 3.10+ with `numpy`, `scipy`, `tqdm`, `colorama`, `cysignals`, `cypari2`.
- PARI/GP (via `cypari2`) — polynomial arithmetic and number field computations.
- Multiprocessing / `concurrent.futures` — parallel residue computation.
- Optional: custom C extension (`collision_walk.c`), which must be compiled manually to `libwalk.so` (the binary is not included in the repository). Without it, DLP mode falls back to slower pure-Python walks or raises a `FileNotFoundError` on import of `walker.py`.

14 Building This Document

This document is built with `latexmk`. A `Makefile` is provided with the following targets:

- `make` or `make all` — compile `docs.tex` to `docs.pdf` using `pdflatex`.
- `make watch` — continuous preview mode: recompiles automatically on every save.
- `make clean` — remove auxiliary files (`.aux`, `.log`, etc.), keeping the PDF.
- `make distclean` — full clean, removing the PDF as well.

Prerequisites: `latexmk` and a standard TeX distribution (e.g. TeX Live or MiKTeX) with `pdflatex`. All packages used (`geometry`, `amsmath`, `booktabs`, `tabularx`, `hyperref`, `enumitem`, `titlesec`) are included in a standard full TeX Live installation.

Module	Description
<code>search7_genus2.sage</code>	Main entry point: orchestrates fibration tower, search, Mumford basis, analyses.
<code>search_common.py</code>	Global configuration and shared utilities (curve setup, prime subgroup, CRT helpers); contains LMFDB test curves.
<code>tower.sage</code>	Fibration tower construction: geometry scoring, polynomial interpolation, tower iteration.
<code>search_main.py</code>	Rational point search: LLL vectors, modular pipeline, CRT, Hensel, rational reconstruction.
<code>mumford_basis.py</code>	Jacobian basis construction: Mumford CRT reconstruction, torsion filter, mod- p certification.
<code>mumford_parallel.py</code>	Parallel residue computation for the Mumford system across the prime pool.
<code>mumford_reconstruction.py</code>	Algebraic pre-filtering and CRT combination of Mumford coordinate residues.
<code>mumford_core.py</code>	Core Mumford divisor arithmetic and group law.
<code>mumford_solver.py</code>	Mumford constraint solving over finite fields.
<code>mumford_equations.py</code>	Mumford condition polynomials.
<code>mumford_height.py</code>	Height computations for Mumford divisors.
<code>mumford_verification.py</code>	Verification routines for reconstructed divisors.
<code>index_calculus.py</code>	Finite-field DLP: factor base generation, relation walk, matrix preparation.
<code>sparse_linalg_modp.py</code>	Block Wiedemann sparse linear algebra solver over $\mathbb{Z}/\ell\mathbb{Z}$.
<code>arakelov.py</code>	Arakelov height pairings (finite places) for Néron–Tate Gram matrix.
<code>homology.py</code>	Period lattice and archimedean height integrals.
<code>periods.py</code>	Period computation for the Jacobian.
<code>integration.py</code>	Numerical integration utilities.
<code>analytic_pairings.py</code>	Analytic/archimedean pairing computations.
<code>pairings.py</code>	Height pairing dispatch.
<code>picard.py</code>	Picard number bounds, Lefschetz trace, Shioda–Tate analysis.
<code>selmer.py</code> / <code>selmer_genus2.py</code>	2-Selmer rank computation for genus 2 curves.
<code>tate.py</code>	Tate algorithm for Kodaira fiber classification and minimal model reduction.
<code>bounds.py</code>	Splitting field degree estimation, height bound auto-tuning, prime pool management.
<code>ll_utilities.py</code>	LLL/BKZ lattice reduction, vector generation, rational reconstruction.
<code>heights.py</code>	Canonical height computations.
<code>torsion.py</code>	Torsion subgroup identification.
<code>automorph.py</code>	Néron–Severi lattice automorphisms.
<code>sat.py</code>	Saturation analysis for the Mordell–Weil group.
<code>nslattice.py</code>	Néron–Severi lattice operations.
<code>local.py</code> / <code>local_diagnostics.py</code>	Local arithmetic and diagnostics.
<code>diagnostics2.py</code>	Singular fiber analysis.
<code>consensus.py</code>	Multi-fibration consensus filter.